

TEMA 1 → CONTROL DE VERSIONES Y DOCUMENTACION

1. Git: El Sistema de Control de Versiones

El **control de versiones** permite registrar y gestionar todos los cambios realizados en archivos de un proyecto. Gracias a él, los desarrolladores pueden recuperar versiones anteriores, revertir errores y trabajar de forma colaborativa sin conflictos.

1.1. Introducción a Git

Git es un sistema de control de versiones **distribuido** creado por Linus Torvalds en 2005.

Diferencias clave respecto a sistemas centralizados:

- Cada desarrollador tiene **una copia completa del repositorio**, incluyendo el historial.
 - Se puede trabajar **sin conexión**.
 - Favorece un desarrollo **no lineal**, con ramas y fusiones de forma ágil.
 - Permite coordinar equipos de forma eficiente y segura.
-

1.2. Conceptos Clave en Git

- **Repositorio:** Contiene todo el historial del proyecto, archivos, ramas y etiquetas.
 - **Commit:** Una “foto” o versión del proyecto. Tiene un mensaje, autor, fecha y un hash único.
 - **Rama (Branch):** Línea independiente de desarrollo para nuevas funciones o correcciones.
 - **Área de Preparación (Staging Area):** Zona donde seleccionas qué cambios entran en el siguiente commit.
 - **Directorio de Trabajo:** Los archivos donde trabajas en tu máquina.
-

1.3. Ciclo de Vida de los Archivos en Git

1. **Modificar archivos** → Cambias algo en tu proyecto.
2. **Staging (git add)** → Seleccionas qué cambios quieres confirmar.
3. **Commit (git commit)** → Guardas esa versión en el repositorio local.

Estados:

- **Modified:** Archivo cambiado, aún no preparado.
 - **Staged:** Listo para el commit.
 - **Committed:** Cambios guardados en el historial del repositorio.
-

2. Plataformas de Colaboración: GitHub y Bitbucket

Permiten trabajar en equipo sincronizando repositorios remotos y gestionando el proyecto.

2.1. Colaboración en la Nube

Comandos básicos:

```
git clone <URL>
git push origen rama
git pull origen rama
```

Conceptos clave:

- **Pull Request:** Solicitud para revisar y fusionar cambios.
 - **Fork:** Copia de un repositorio para contribuir sin afectar el original.
-

2.2. Flujos de Trabajo

GitFlow:

Es el flujo más completo y recomendado.

- Ramas principales: **main/master** y **develop**.
- Ramas temporales:
 - *feature* (nuevas funcionalidades)
 - *release* (versiones)
 - *hotfix* (errores urgentes)

Lo importante: **acordar un flujo y seguirlo siempre.**

2.3. Gestión de Proyectos con Scrum

Las plataformas como GitHub o Bitbucket facilitan Scrum mediante:

Issues:

- Historias de usuario → Funcionalidades vistas por el usuario.
- Tareas → Actividades concretas.
- Bugs → Errores del software.

Pueden tener responsables, etiquetas, prioridades...

Tableros de proyecto (Kanban/Scrum):

- Pendiente
- En progreso
- En revisión
- Completado

Wikis:

Guías, manuales, instalación, arquitectura.

Confluence:

Documentación avanzada, actas, diagramas y trabajo colaborativo.

2.4. Flujo Profesional: Del Commit a la Tarea (Issue)

Cada commit debe estar vinculado a un issue para mantener **trazabilidad**.

Ejemplo:

```
git commit -m "feat: Añadido login (#15)"
```

Palabras clave para cerrar issues automáticamente:

- closes #15
 - fixes #15
 - resolves #15
-

2.5. Herramientas Complementarias

- **VS Code:** Integración Git muy completa.
 - **GitKraken:** Cliente visual avanzado.
 - **GitHub Desktop:** Cliente oficial gráfico de GitHub.
-

3. Documentación de Código

Documentar el código permite entenderlo, mantenerlo y ampliarlo en el futuro, evitando depender de la memoria o del desarrollador original.

3.1. Markdown

Markdown es un lenguaje de marcado simple para documentación técnica.

Ventajas:

- Fácil lectura/escritura.
- Muy usado en GitHub para README y wikis.
- Archivos portables (texto plano).
- Soporte universal.

Sintaxis básica:

- Encabezados: #, ##, ###, etc.
 - Negrita: **texto**
 - Cursiva: *texto*
 - Listas: -, +, *
 - Enlaces: [texto](url)
 - Imágenes: ![texto](ruta)
 - Código en bloque:
``java
código
 - Tablas con | y -.
-

3.2. Javadoc (Java)

Javadoc genera documentación HTML a partir de comentarios en el código Java.

Bloques Javadoc:

Empiezan por `/**` y terminan con `*/`.

Etiquetas importantes:

Etiqueta	Descripción	Ejemplo
<code>@param</code>	Describe un parámetro de un método o constructor.	<code>@param id</code> El identificador único del usuario.
<code>@return</code>	Describe el valor que un método retorna.	<code>@return</code> El objeto Usuario si se encuentra.
<code>@throws</code>	Documenta una excepción que un método puede lanzar y la condición para ello.	<code>@throws IllegalArgumentException</code> si el ID es nulo.
<code>@author</code>	Indica quién escribió la clase.	<code>@author</code> TuNombre
<code>@version</code>	Especifica la versión actual de la clase o API.	<code>@version 1.0</code>
<code>@deprecated</code>	Marca una clase o método como obsoleto.	<code>@deprecated</code> Usa el método <code>findById()</code>
<code>@see</code>	Proporciona un enlace de referencia a otra parte del código o a una URL externa.	<code>@see</code> <code>UserService#findUserById(Long)</code>
<code>{@link}</code>	Crea un enlace en línea a otro elemento del código, dentro del texto.	Retorna un objeto <code>{@link Optional}</code> .

Soporta HTML: `<p>`, `<b>`, `<pre>`, etc.

Generar documentación:

Estos dos comandos:

- `javadoc -d docs ruta/al/código`
- `mvn javadoc:javadoc`

En IDE:

- IntelliJ: Tools → Generate Javadoc
- Eclipse: Project → Generate Javadoc

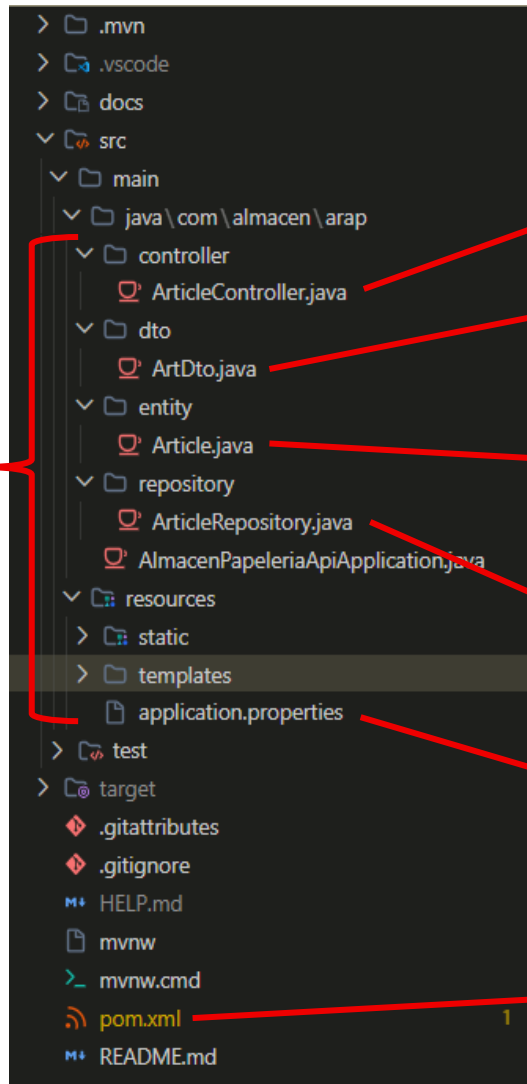
Resultado:

Documentación profesional, legible y navegable para cualquier desarrollador.

Estrucctura de proyecto con Spring boot

Estrucctura:

- **src/main/java/com/almacen/arap** (estructura que te genera el [Spring Initializr](#))
- **controller**
- **dto** (clase simplificada de la entidad)
- **entity** (donde se aloja la clase principal)
- **repository** (se comunica con la base de datos)
- **resources** (ya viene por defecto)



- **Controlador REST** que maneja las peticiones HTTP (endpoints).
- Suele ser una versión simplificada de la entidad. Evita exponer la entidad completa al exterior
- **Entidad JPA** que representa la tabla en la base de datos. Define la estructura de los artículos (campos, relaciones)
- **Interfaz de repositorio** que extiende de `JpaRepository`. Se comunica con la base de datos
- Define la **configuración de la aplicación Spring Boot**, como conexión a base de datos y parámetros del servidor.
- Archivo de configuración de Maven que gestiona las dependencias, plugins y configuración del proyecto, para que funcione correctamente